

Now consider an `addToTotal()` function, which adds the numeric argument to a global variable named `TOTAL`:

```
>>> TOTAL = 0
>>> def addToTotal(amount):
...     global TOTAL
...     TOTAL += amount
...     return TOTAL
...
>>> addToTotal(10)
10
>>> addToTotal(10)
20
>>> addToTotal(9999)
10019
>>> TOTAL
10019
```

The `addToTotal()` function does have a side effect, because it modifies an element that exists outside of the function: the `TOTAL` global variable. Side effects can be more than changes to global variables. They include updating or deleting files, printing text onscreen, opening a database connection, authenticating to a server, or making any other change outside of the function. Any trace that a function call leaves behind after returning is a side effect.

Side effects can also include making in-place changes to mutable objects referred to outside of the function. For example, the following `removeLastCatFromList()` function modifies the list argument in-place:

```
>>> def removeLastCatFromList(petSpecies):
...     if len(petSpecies) > 0 and petSpecies[-1] == 'cat':
...         petSpecies.pop()
...
>>> myPets = ['dog', 'cat', 'bird', 'cat']
>>> removeLastCatFromList(myPets)
>>> myPets
['dog', 'cat', 'bird']
```

In this example, the `myPets` variable and `petSpecies` parameter hold references to the same list. Any in-place modifications made to the list